Fire-Sale Cascade Simulator · Case 02 · ארכיטקטורה:

פרק 3 (ארכיטקטורה) בתבנית תיק הפרויקט של מגשימים: מבט על, התאמה לאפיון, ועיצוב נתונים וישויות מידע. נוספו שלושה סעיפים שאינם בתבנית ומשמשים את שלב ההשוואה: דפוסים ארכיטקטוניים ופשרות, תרחישי כשל וסדר בנייה, ובדיקת סגירה.

מבט על

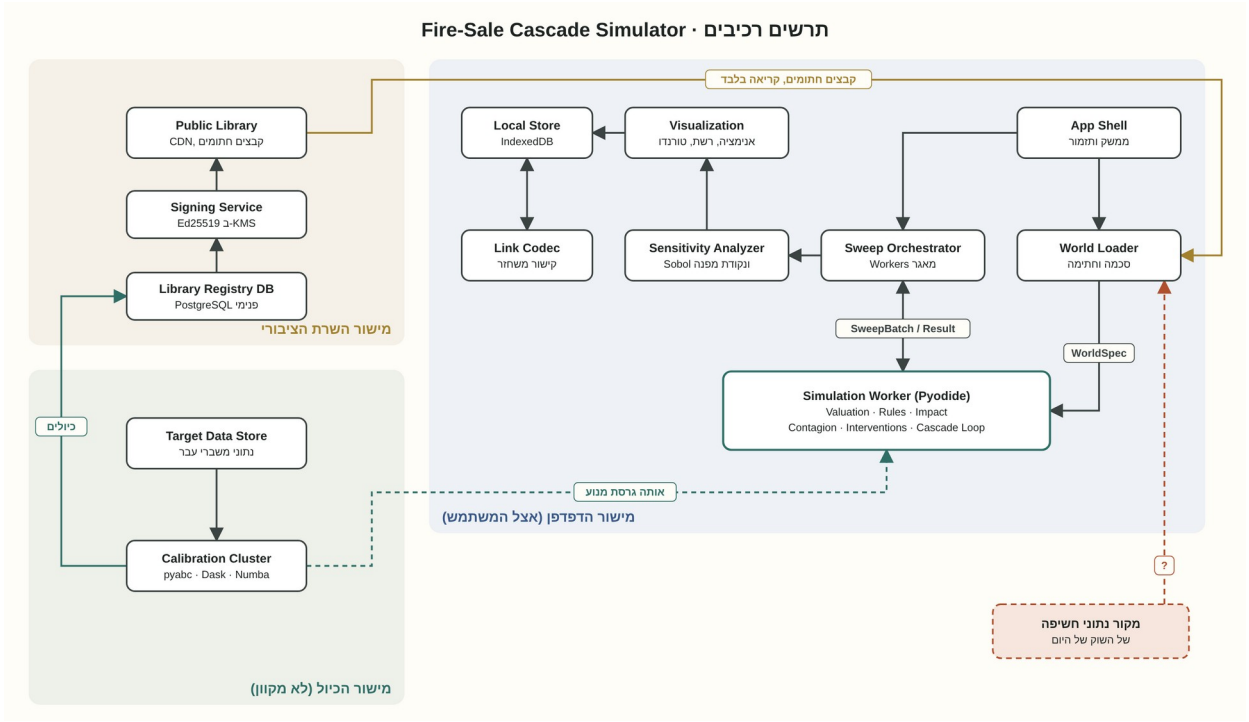
העיקרון שמארגן את כל המבנה

מודל האיום קבע שנתוני מאזן שמשתמש טוען הם מידע רגיש, ולכן החישוב רץ אצל המשתמש. מכאן נגזרת חלוקה לשלושה מישורים (planes), וכל רכיב שייך לאחד מהם בלבד:

- **מישור הדפדפן:** כל מה שנוגע בעולם ובתוצאות של משתמש. רץ במחשב שלו, ולא שולח החוצה נתונים.
- **מישור השרת הציבורי:** ספרייה של קבצים סטטיים חתומים (עולמות, כללים, כיולים). מקבל בקשות קריאה בלבד.
- **מישור הכיול:** אשכול חישוב שרץ לא מקוון, מייצר את הכיולים, ומפרסם אותם לשרת הציבורי.

אין ערוץ שבו מידע זורם מהדפדפן לשרת. זו לא רק מדיניות: היא נאכפת בכותרת CSP, שחוסמת בדפדפן כל בקשה שאינה קריאה מהשרת שלנו.

תרשים רכיבים



הרכיבים

- **App Shell (מישור הדפדפן):** האפליקציה עצמה. מנהלת את המסכים, מקבלת קלט מהמשתמש, ומעבירה עבודה לרכיבים שרצים ברקע. כתובה ב-TypeScript, בלי framework כבד.
- **World Loader:** טוען עולם ממקור אחד מתוך שלושה (עולם רשמי מהספרייה, הגדרת התפלגויות, או קובץ פרטי), מאמת אותו מול הסכמה, ובודק חתימה לעולם רשמי. מייצר WorldSpec ומאזנים התחלתיים.

- **Simulation Worker:** Web Worker שמריץ Pyodide עם NumPy, ובתוכו מנוע הסימולציה. שישה תתי-רכיבים: Valuation (שערוך מאזנים לפי מחירים), Rules Interpreter (מפעיל את כללי התגובה שנטענו כנתונים), Market Impact (מתרגם מכירות לתנועת מחיר), Contagion (הדבקה ישירה ו-DebtRank), Interventions (מפעיל התערבויות לפי טריגרים), ו-Cascade Loop (הלולאה שמחברת את כולם ומחזירה CascadeResult).
- **Sweep Orchestrator:** מחלק סריקה של מאות עולמות בין כמה Simulation Workers במקביל (לפי מספר הליבות במחשב), אוסף תוצאות, ומעביר אותן לניתוח.
- **Sensitivity Analyzer:** מחשב מדדי Sobol, אחוזונים ונקודות מפנה מתוך תוצאות הסריקה.
- **Visualization:** אנימציית המפל (Canvas), מפת הרשת (d3-force), תרשים הטורנדו (SVG).
- **Link Codec:** מקודד ומפענח הרצה לקישור משחזר (בחלק ה-fragment של הכתובת, שלא נשלח לשרת).
- **Local Store:** IndexedDB. היסטוריית הרצות, סריקות ועולמות פרטיים.
- **Public Library (מישור השרת):** CDN עם קבצים סטטיים: סוגי סוכנים, ערכות כללים, עולמות רשמיים וחתימותיהם, כיולים ודוחות התאמה, ומפתח האימות הציבורי.
- **Library Registry DB:** PostgreSQL שבו מנוהלת הספרייה לפני פרסום. לא נגיש מבחוץ: הוא מייצא קבצים לקטלוג הציבורי.
- **Signing Service:** חותם על כל עולם רשמי וקובץ כיול לפני פרסום. המפתח הפרטי ב-KMS.
- **Calibration Cluster (מישור הכיול):** Dask עם pyabc ו-Numba. מריץ את אותו מנוע סימולציה (אותה גרסה בדיוק), מול נתוני יעד ממשברי עבר, ומייצר התפלגות פרמטרים ודוח התאמה.
- **Target Data Store:** נתוני היעד לכיול, עם מקור לכל מספר (דוחות ה-Bank of England, הפד ו-SVB).

טבלת רכיבים: תפקיד וממשקים

רכיב	תפקיד	ממשקים נכנסים	ממשקים יוצאים
App Shell	ניהול מסכים, קלט משתמש, תזמור עבודה	פעולות משתמש, קישור משחזר	בקשות ל-World Loader, ל-Sweep Orchestrator ול-Visualization
World Loader	טעינה, אימות סכמה, בדיקת חתימה	קובץ עולם מהספרייה, קובץ פרטי, הגדרת התפלגויות	WorldSpec ומאזנים התחלתיים
Simulation Worker	הרצת מפל מלא על עולם אחד	RunRequest (עולם, זעזוע, התערבויות, seed)	CascadeResult
Sweep Orchestrator	חלוקת סריקה בין Workers ואיסוף	SweepSpec	CascadeResult אוסף לניתוח
Sensitivity Analyzer	Sobol, אחוזונים, נקודות מפנה	אוסף תוצאות ופרמטרים	SweepResult
Visualization	אנימציה, רשת, טורנדו, כיתוב נגיש	CascadeResult, SweepResult, ExposureGraph	תצוגה ותמונה להורדה
Link Codec	קידוד ופענוח הרצה לקישור	RunLink	כתובת עם fragment
Local Store	שמירה מקומית	תוצאות ועולמות פרטיים	שליפה להיסטוריה
Public Library	הגשת קבצים סטטיים חתומים	בקשות GET בלבד	קבצי JSON, Parquet וחתימות
Library Registry DB	ניהול הספרייה לפני פרסום	עדכונים מצוות הפרויקט, כיולים	ייצוא לקבצים
Signing Service	חתימה על תוכן רשמי	קובץ קנוני	חתימת Ed25519
Calibration Cluster	כיול מול משברי עבר	נתוני יעד, התפלגויות מוקדמות, מנוע	התפלגות פרמטרים, דוח התאמה
Target Data Store	נתוני יעד עם מקורות	הזנה ידנית מדוחות	נתוני יעד לכיול

רכיב	תפקיד	ממשקים נכנסים	ממשקים יוצאים
		ציבוריים	

התאמה לאפיון

פיצ'ר מהאפיון	רכיבים מעורבים	סדר פעולות ולמה
1. בניית עולם וספריית סוכנים	App Shell ← World Loader ← Public Library	הבדיקה של החתימה קורית לפני שהעולם מועבר למנוע, כך שעולם שנכשל בבדיקה לעולם לא מסומן "רשמי"
2. מאזן ושערוך	Simulation Worker (Valuation)	רץ בתחילת כל סבב, לפני הכללים, כי הכללים מחליטים לפי ההון הכלכלי העדכני
3. כללי תגובה	Public Library (ערכות כללים) ← World Loader ← Simulation Worker (Rules Interpreter)	הכללים נטענים כנתונים ומהודרים פעם אחת לפעולות וקטוריות, לא בכל סבב
4. השפעת מחיר	Simulation Worker (Market Impact)	רץ אחרי ההתערבויות, כדי שספיגה של הבנק המרכזי תקטין את המכירות לפני שהן מזיזות מחיר
5. מנוע המפל	Simulation Worker (Cascade Loop)	הלולאה עצמה: שערוך, כללים, סליקה, התערבות, השפעת מחיר, הדבקה, בדיקות שימור
6. רשת והדבקה	World Loader (בניית הגרף) ← Simulation Worker (Contagion)	DebtRank מחושב פעם אחת לפני ההרצה (מבנה), וההדבקה בכל סבב (דינמיקה)
7. התערבות	App Shell (הגדרה) ← Simulation Worker (Interventions)	ההכרזה משפיעה על מקדם ההשפעה מהסבב שבו היא מוכרזת, והרכישות מהסבב הבא
8. סריקה ורגישות	App Shell ← Sweep Orchestrator ← Simulation Workers (במקביל) ← Sensitivity Analyzer	הרצה בודדת מנותבת אוטומטית גם לסריקה ברירת מחדל, כך שאין תוצאה בלי טווח
9. שחזור וכיול	Target Data Store ← Calibration Cluster ← Signing Service ← Public Library	הכיול לא רץ אצל המשתמש. הוא מתפרסם כקובץ חתום, והעולמות הרשמיים מפנים אליו
10. תצוגה וקישור	Visualization, Link Codec, Local Store	הקישור נוצר רק לעולם שאינו פרטי, וה-fragment לא נשלח לשרת

עיצוב נתונים וישויות מידע

מידע שעובר בין רכיבים (לא נשמר)

- **RunRequest, מ-App Shell או מ-Sweep Orchestrator אל Simulation Worker:** מזהה עולם או המאזנים עצמם (כ-Transferable ArrayBuffer, בלי העתקה), זעזוע, רשימת התערבויות, seed, וגרסת מנוע.
- **CascadeResult, מ-Simulation Worker אל ה-Orchestrator או ל-App Shell:** מערכי RoundState דחוסים (מערכים טיפוסיים), ומדדים מסכמים.
- **SweepBatch, מ-Sweep Orchestrator ל-Worker:** רשימת וקטורי פרמטרים ממקטע של רצף Sobol, כדי שכל Worker יקבל חלק נפרד ושלם.
- **ProgressEvent, מה-Workers ל-App Shell:** כמה הרצות הסתיימו, להצגת התקדמות ולביטול.

מידע שנשמר

- **בשרת הציבורי (קבצים סטטיים, לקריאה בלבד):** agent_types, rule_sets, worlds עם חתימות ומקורות, calibrations (התפלגות ב-Parquet ודוח התאמה ב-JSON), ומפתח ציבורי.
- **ב-Library Registry DB (פנימי):** אותן ישויות בגרסת העבודה, עם היסטוריית שינויים. זו הטבלה שממנה מייצאים.
- **ב-Calibration Cluster:** נתוני יעד, הרצות כיול, והתפלגויות. פנימי, ומתפרסם רק דרך Signing Service.
- **בדפדפן (IndexedDB):** runs, sweeps, private_worlds. מה שנשמר כאן לעולם לא מגיע לשרת.

הפרדה מכוונת

התוצאה של הרצה (CascadeResult) והקישור אליה (RunLink) הן שתי ישויות נפרדות. הקישור מכיל רק את מה שצריך כדי לשחזר את ההרצה, לא את התוצאה. כך גם קישור שדלף מגלה רק הגדרה, ואף פעם לא נתוני מאזן של משתמש.

דפוסים ארכיטקטוניים ופשרות

ליבה פונקציונלית, מעטפת אימפרטיבית (Functional Core, Imperative Shell)

מנוע הסימולציה הוא פונקציה טהורה: אותו עולם, אותו זעזוע ואותו seed יחזירו תמיד אותה תוצאה בדיוק. אין לו גישה לרשת, לזמן או לאקראיות חיצונית. כל מה שיש לו "תופעות לוואי" (קריאת קבצים, שמירה, ציור) נמצא במעטפת, ב-App Shell וב-Local Store.

- **מה מרוויחים:** שחזור מלא (הקישור המשחזר עובד רק בגלל זה), בדיקות פשוטות, ואותו מנוע בדיוק בדפדפן ובאשכול הכיול.
- **מה משלמים:** צריך להעביר למנוע את כל המצב כקלט ולקבל אותו כפלט, וזה יותר העתקות זיכרון. אנחנו מצמצמים את זה עם מערכים טיפוסיים שמועברים בלי העתקה.

עיצוב מונחה נתונים (Data-Oriented Design), בהשראת ECS ממנועי משחקים

במקום אובייקט לכל סוכן עם מתודות, כל תכונה היא מערך של כל הסוכנים (Structure of Arrays), וכל "מערכת" (שערוך, כללים, השפעה) היא פונקציה שעוברת על המערכים. כך עובדים מנועי משחקים שמנהלים מיליוני ישויות.

- **מה מרוויחים:** ה-POC מדד 1,000 הרצות ב-0.28 שניות. בסימולציה עם אובייקט לכל סוכן זה היה איטי פי עשרות.
- **מה משלמים:** קוד פחות "טבעי" לקריאה. סוכן אינו אובייקט שאפשר להחזיק ביד, אלא אינדקס במערכים. לכן ממשק הכללים (פיצ'ר 3) כתוב במונחים של סוכן בודד, ומתורגם לפעולות על מערכים.

כללים כנתונים (Interpreter Pattern)

כללי התגובה הם קובצי YAML שנטענים ומתורגמים לפעולות. המנוע לא יודע מה זה "קרן LDI", הוא יודע להפעיל תנאי ופעולה.

- **מה מרוויחים:** אנליסט יכול לקרוא ולשנות כלל, ושתי ערכות כללים ניתנות להשוואה על אותו עולם.
- **מה משלמים:** שפת הכללים מוגבלת בכוונה. כלל שלא ניתן לבטא בה דורש שינוי במנוע. וצריך לוודא שהמתרגם לא מריץ קוד שרירותי מקובץ של משתמש (אין eval).

פיזור ואיסוף בדפדפן (Fork-Join) על מאגר Workers

סריקה מחולקת לפי מקטעים של רצף Sobol בין כמה Workers, וכל Worker מחזיר תוצאות חלקיות.

- **מה מרוויחים:** סריקה של 512 עולמות על מחשב עם 8 ליבות רצה פי כ-6 מהר יותר.

- **מה משלמים:** כל Worker טוען Pyodide בנפרד (כ-10 מגה-בייט וכמה שניות בפעם הראשונה). לכן המאגר נוצר פעם אחת ונשאר חי לאורך כל השימוש.

פרסום בלבד לשרת (Read-Only Publication)

השרת הציבורי לא מקבל כתיבה מאף משתמש. כל התוכן בו נוצר בתהליך פנימי, נחתם, ומתפרסם כקבצים.

- **מה מרוויחים:** משטח תקיפה זעיר (אין טפסים, אין API לכתיבה, אין מסד נתונים חשוף), ואפשר להגיש הכול מ-CDN.
- **מה משלמים:** אין שיתוף בין משתמשים מתוך המערכת. מי שרוצה לשתף שולח קישור.

תרחישי כשל

מה נכשל	מה קורה	למה זה בסדר (או לא)
ה-CDN לא זמין	עולמות רשמיים לא נטענים. עולמות מהתפלגויות ופרטיים עובדים מהמטמון המקומי	המנוע רץ מקומית, אז הכלי עובד חלקית גם בלי שרת
חתימה של עולם לא תקינה	העולם נטען כ"מותאם אישית", עם אזהרה בולטת	אי אפשר להציג תוצאה כרשמית בלי חתימה תקינה
Worker קורס באמצע סריקה	ה-Orchestrator מזהה, מריץ מחדש את המקטע ב-Worker אחר	ההרצות דטרמיניסטיות, אז ריצה חוזרת נותנת אותה תוצאה
המפל לא מתכנס אחרי 100 סבבים	התוצאה מסומנת "לא התכנס" ולא מוצגת כמספר	ספירלה שלא נגמרת היא ממצא, לא באג, וצריך לראות אותה ככזו
בדיקת שימור נכשלת	ההרצה נעצרת עם שגיאה ופרטים	עדיף שגיאה מתוצאה שגויה שנראית סבירה
הכיול לא משחזר נתון יעד	הדוח מסמן אותו באדום, והעולם הרשמי מפרסם את זה	ההגבלה גלויה, לא מוסתרת

סדר בנייה

- **ספרינט 1, מנוע מינימלי:** Valuation, Cascade Loop, ושני סוגי סוכנים (LDI ובנק) בפייתון רגיל, עם בדיקות שימור. מטרה: לשחזר את ה-POC במבנה החדש.
- **ספרינט 2, כללים והשפעה:** Rules Interpreter עם YAML, שלושת מודלי ההשפעה, וחמשת סוגי הסוכנים.
- **ספרינט 3, דפדפן:** העברת המנוע ל-Pyodide בתוך Worker, World Loader, ו-App Shell בסיסי עם גרף תשואה.
- **ספרינט 4, סריקה:** Sweep Orchestrator, Sensitivity Analyzer, ותרשים הטורנדו.
- **ספרינט 5, הדבקה והתערבות:** Contagion, DebtRank, וארבעת סוגי ההתערבות.
- **ספרינט 6, ספרייה וכיול:** Library Registry, Signing Service, Calibration Cluster, ושני העולמות הרשמיים הראשונים.
- **ספרינט 7, ליטוש:** אנימציה, מפת רשת, קישור משחזר, Local Store, CSP ונגישות.

בדיקת סגירה

השאלה של בדיקת הסגירה פשוטה: האם המערכת יכולה לקיים את ההבטחה המרכזית שלה מקצה לקצה, רק עם רכיבים שנמצאים בתוך הגבולות שלה? ההבטחה המרכזית של הפרויקט, מהייזום: לראות משבר שמגיע לאט, לפני שהוא מגיע מהר. כלומר, לומר משהו על השוק כפי שהוא היום.

רכיב או קלט	בתוך הגבולות?	האם ההבטחה תלויה בו?
מנוע הסימולציה (כל תתי-הרכיבים)	כן	כן, ונמצא בשליטה מלאה
סריקה ורגישות	כן	כן, ונמצא בשליטה מלאה
עולמות רשמיים של משברי עבר	כן (מכוילים מדוחות ציבוריים)	רק לשחזור העבר
נתוני יעד לכיול	כן (מדוחות ציבוריים)	רק לכיול
התפלגות המנוף והחשיפות בשוק של היום	לא	כן, וזה הקלט המכריע

השורה האחרונה היא הבעיה. ה-POC הראה, והסריקה בפיצ'ר 8 תמדוד את זה בכל הרצה מחדש, שבין מנוף ממוצע של 4 ל-5.5 התוצאה עוברת מ"שקט" ל"ספירלה כמעט ודאית". את הנתון הזה המערכת לא מייצרת ולא יכולה לייצר: היא יכולה רק לנחש טווח, או לקבל אותו ממקור חיצוני. בתרשים הרכיבים זה מסומן כקופסה מקווקוות מחוץ לגבולות: "מקור נתוני חשיפה של השוק". אין רכיב שממלא אותה.

מה המערכת כן מבטיחה, במלואה ובתוך הגבולות שלה: "אם המנוף והחשיפות הם X, זה מה שיקרה, וזה כמה התשובה רגישה ל-X". זו תשובה מותנית. היא שימושית מאוד ללמידה ולשחזור משברי עבר. אבל על השאלה שפתחה את הפרויקט, מה מצב השוק עכשיו, התשובה שלה עוברת דרך קופסה שאין לה.